

Assignment_04_T1_1_2

December 22, 2023

1 Authors

- Raphael Wäspi (18-918-938)
- Nicolas Huber (16-936-205)

```
[140]: import math
# Imports
import numpy as np
import matplotlib.pyplot as plt
import networkx as nx
import os
import glob
import re
import random
from statistics import mean
import pandas as pd
from typing import Iterable, List, Tuple

import scipy.special
from tqdm import tqdm
import shutil
from joblib import Parallel, delayed
from scipy.special import factorial
import powerlaw
from NEMtropy import DirectedGraph, UndirectedGraph
from NEMtropy import matrix_generator as mg
from NEMtropy.network_functions import build_adjacency_from_edgelist
```

```
[141]: # Define colors with reduced opacity from a colormap
COLOR_BLUE = 'dodgerblue'
COLOR_GREEN = 'palegreen'
COLOR_RED = 'lightcoral'
COLOR_YELLOW = 'gold'
COLOR_GREY = 'slategrey'
COLOR_BLACK = 'black'
COLOR_LIGHT_GRAY = 'lightgray'
COLOR_DARK_GRAY = 'darkgray'
```

```
# Define the main data directory
OUTPUT_PATH = "./output"
```

[142]: # Prepare output folder

```
# Delete the entire 'output' directory and its contents
if os.path.exists(OUTPUT_PATH):
    shutil.rmtree(OUTPUT_PATH)

# Create the 'output' directory
os.makedirs(OUTPUT_PATH)

# Define subdirectories
### OUTPUT_PATH_ERGMS = os.path.join(OUTPUT_PATH, "ERGMS")

# Recreate the subdirectories
### os.makedirs(OUTPUT_PATH_ERGMS)
```

[143]: # Define the directory containing the .gml files

```
directory = 'datasets'

# Use glob to get a list of all .gml files in the directory
gml_files = glob.glob(os.path.join(directory, '*.gml'))

# Init an empty dictionary for all graphs - the key is the actual name, and the
↳ value is the graph data
graphs = {}

# Use tqdm to show a progress bar in Jupyter Notebook
with tqdm(total=len(gml_files), desc="Loading GML Files") as pbar:
    # Loop through the list of .gml files and load them
    for file_path in gml_files:
        with open(file_path, 'r') as file:
            content = file.read()

        # Use regex to extract the desired part
        match = re.search(r'/(~/+)\.gml$', file_path)
        if match:
            graph_name = match.group(1)

            # Add to the graph dictionary
            graph = nx.read_gml(file_path)
            graphs[graph_name] = graph

        # Print the graph name when each part is finished
        print(f"Finished loading {graph_name}")
```

```
pbar.update(1) # Update the progress bar
```

```
Loading GML Files: 100%|          | 4/4 [00:00<00:00, 25.70it/s]
```

```
Finished loading graph1.2
```

```
Finished loading graph_madrid
```

```
Finished loading graph_jazz_collab
```

```
Finished loading graph1.1
```

2 1. Compartmental Models on Networks

2.1 1.1

Write a function to simulate the SI dynamic on a generic network and provide a visualisation. The function should take in input: - a network, e.g. if you are using networkx a generic `nx.Graph`, - the infection rate β , - an infected seed node.

You may also want to add other parameters, e.g. the maximum number of steps for the simulation (some bad things may happen if you don't). As an output, the function should return the list of nodes infected in the simulation. Additionally, run the SI function for an adequate number of steps, with $\beta = 0.5$ on the following two graphs (the datasets are provided), with infected seed node "10":
- graph1.1.gml - graph1.2.gml

and provide a visualisation of the final state of the epidemic, where all the infected nodes at the end of the run are red, or blue if they are not infected.

```
[148]: def si_sim(graph, beta, seed_node, max_steps=100):  
    # Set of infected nodes with the seed node  
    infected_nodes = set()  
    infected_nodes.add(seed_node)  
  
    nodes = set(graph.nodes) # Set of all nodes in the graph  
    steps = 0  
  
    while steps < max_steps:  
        infected_nodes_copy = infected_nodes.copy()  
        susceptible_nodes = nodes - infected_nodes_copy # Identify susceptible  
        nodes by subtracting infected nodes from all nodes  
  
        for node in susceptible_nodes:  
            neighbors = set(graph.neighbors(node)) # Get neighbors of current  
            node  
            infected_neighbors = neighbors & infected_nodes_copy # Get infected  
            neighbors of current node  
  
            num_infected_neighbors = len(infected_neighbors)
```

```

        if num_infected_neighbors > 0:
            probb_healthy = (1-beta)**num_infected_neighbors
            probb_infected = 1 - probb_healthy
            if random.random() < probb_infected:
                infected_nodes.add(node) # Node gets infected

    steps += 1

    return list(infected_nodes)

def visualize_epidemic(graph, infected_nodes):
    pos = nx.spring_layout(graph)

    # Draw nodes
    nx.draw_networkx_nodes(graph, pos, nodelist=infected_nodes,
    ↪node_color='red', label='Infected')
    nx.draw_networkx_nodes(graph, pos, nodelist=set(graph.nodes) -
    ↪set(infected_nodes), node_color='blue', label='Susceptible')

    # Draw edges
    nx.draw_networkx_edges(graph, pos)

    # Draw labels
    nx.draw_networkx_labels(graph, pos)

    # Display legend
    plt.legend()

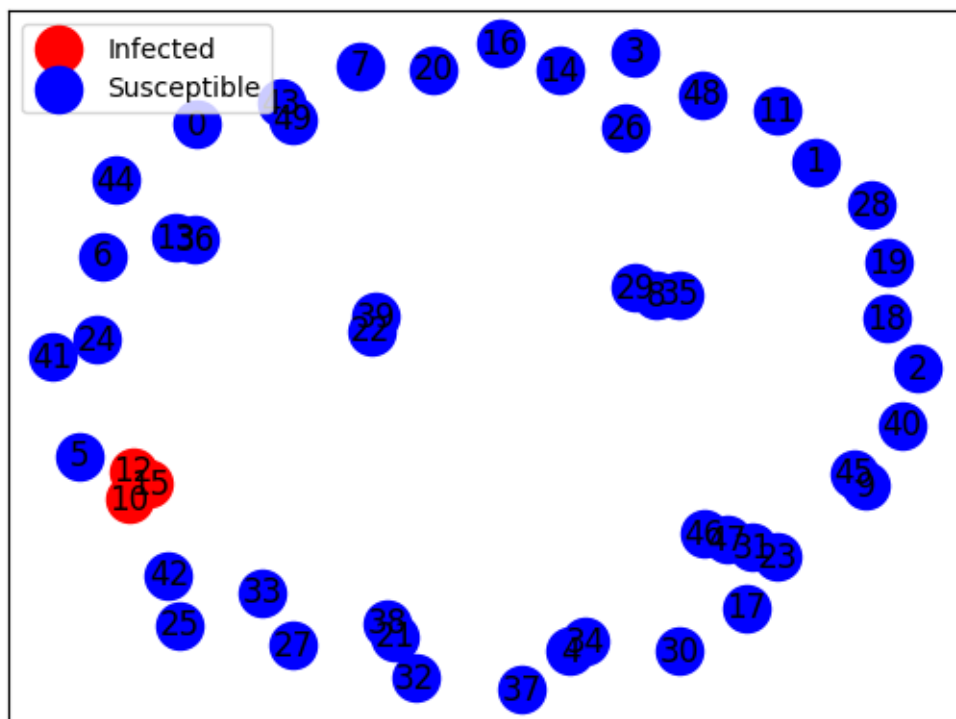
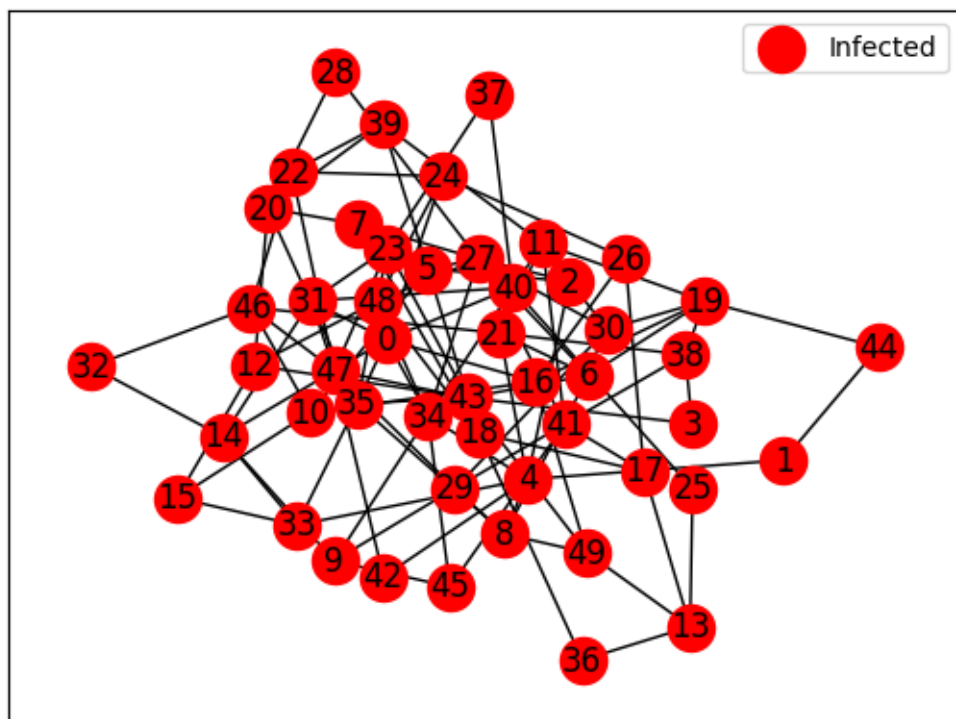
    # Show the plot
    plt.show()

graph1_1 = graphs["graph1.1"]
graph1_2 = graphs["graph1.2"]
beta = 0.5
seed_node = "10"

infected_nodes1_1 = si_sim(graph1_1, beta, seed_node)
visualize_epidemic(graph1_1, infected_nodes1_1)

infected_nodes1_2 = si_sim(graph1_2, beta, seed_node)
visualize_epidemic(graph1_2, infected_nodes1_2)

```



2.2 1.2

To extend the function you wrote for point 1 in order to simulate the SIS model. You need to introduce γ , i.e. the recovery rate, to the input parameters of the function.

Provide a plot of $\hat{I}$ and $\hat{S}$ as a function of the simulation time-step, on graph `graph1.1.gml` with $\beta = 0.2$ and $\gamma = 0.4$ over 1000 steps (at least). Choose a random seed such that you don't end up in the trivial stationary states $(\hat{I}, \hat{S}) = (0, 1)$

Hint: in order to observe “smoother” result you can use a moving average. You can check out moving average functions on `pandas` python module, or implement it yourself.

```
[149]: def sis_sim(graph, beta, gamma, seed_node, max_steps=100):
    infected_nodes = {seed_node}
    nodes = set(graph.nodes)

    infected_ratios = []
    susceptible_ratios = []

    step = 0
    while step < max_steps:
        newly_infected = set()
        newly_recovered = set()

        for infected_node in infected_nodes:
            for neighbor in graph.neighbors(infected_node):
                if neighbor not in infected_nodes and np.random.uniform(0, 1) < beta:
                    newly_infected.add(neighbor)
                if np.random.uniform(0, 1) < gamma:
                    newly_recovered.add(infected_node)

        infected_nodes.update(newly_infected)
        infected_nodes.difference_update(newly_recovered)

        infected_ratio = len(infected_nodes) / len(nodes)
        susceptible_ratio = 1 - infected_ratio

        infected_ratios.append(infected_ratio)
        susceptible_ratios.append(susceptible_ratio)

        step += 1

    return infected_ratios, susceptible_ratios

def moving_average(lst, window_size):
    return pd.Series(lst).rolling(window=window_size).mean()

# Example
```

```

graph1_1 = graphs["graph1.1"]

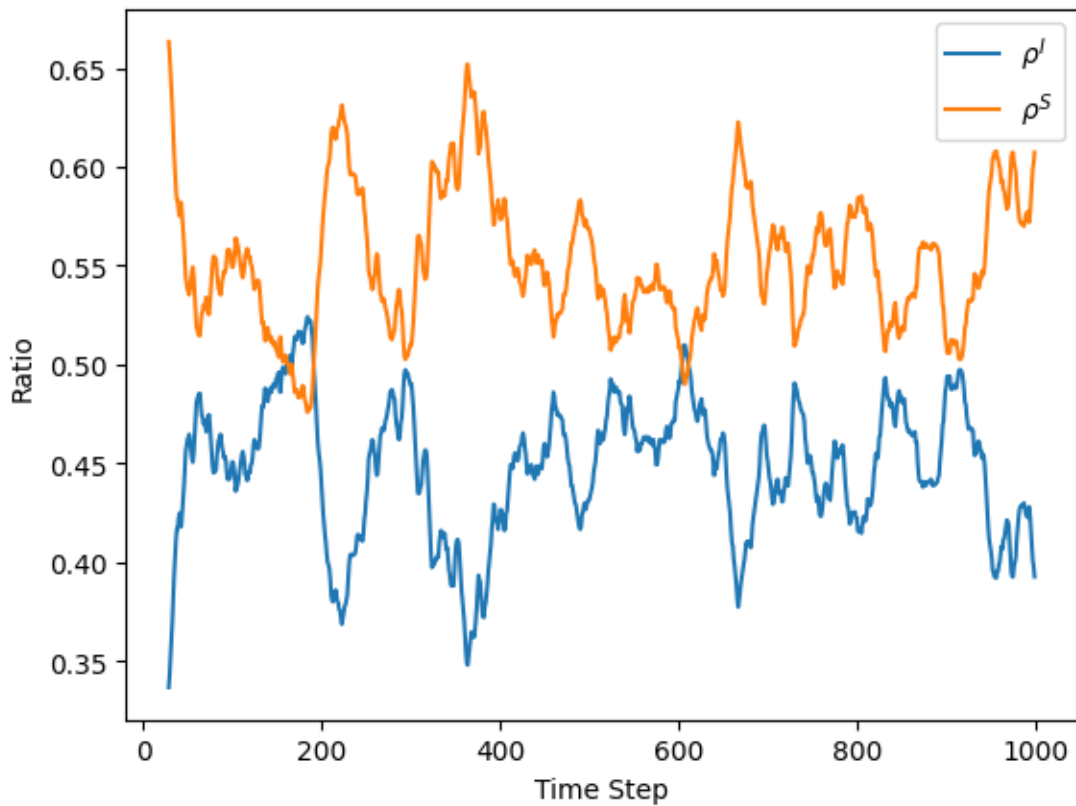
beta = 0.2
gamma = 0.4
seed_node = np.random.choice(list(graph1_1.nodes))
max_steps = 1000

# Run the SIS simulation
infected_ratios, susceptible_ratios = sis_sim(graph1_1, beta, gamma, seed_node,
↪max_steps)

# Plot infected and susceptible ratios with moving average
window_size = 30
infected_ratios_smooth = moving_average(infected_ratios, window_size)
susceptible_ratios_smooth = moving_average(susceptible_ratios, window_size)

plt.plot(infected_ratios_smooth, label=r'$\rho^I$')
plt.plot(susceptible_ratios_smooth, label=r'$\rho^S$')
plt.xlabel('Time Step')
plt.ylabel('Ratio')
plt.legend()
plt.show()

```



2.3 1.3

Observe how the infected share of the population stationary state is affected by the network topology, in the SIS model. You have to plot the stationary state share of infected nodes $\hat{I}(t = \infty)$ as a function of β/γ , on two different kind of network structures: - For the Erdos-Renyi $N = 500$ and $p = 6/N$ - For the Barabasi-Albert $N = 500$ and $m = 3$

You can use networkx native functions to create the networks. You need to average $\hat{I}$ on multiple SIS runs, a sample size of 100 SIS simulations per β/γ combination will work. For the x-axis of the plot β/γ chose a logarithmic scale with at least 10 values in the interval $[0, 1]$.

On the y-axis you need to plot: - $\hat{I}_{ER}(t = \infty)$, the share of infected nodes in the stationary state for the Erdos-Renyi network, averaged over at least 100 simulations as a function of β/γ - ER critical epidemic threshold RC. - $\hat{I}_{BA}(t = \infty)$, the share of infected nodes in the stationary state for the Barabasi-Albert network, averaged over at least 100 simulations as a function of β/γ - BA critical epidemic threshold RC.

Hint: It may take a while to run the simulations, take this into account.

Look at task_1.3separated.ipynb

2.4 1.4

Observe the effect of different immunisation strategies. Modify your SIS function to introduce the immunised state, a state where the node cannot get infected nor spread any more. Then implement two different immunised strategies:

- Random strategy: immunised a fraction $g \in [0, 1]$ of the population, where the nodes immunised are chosen randomly.
- Target Strategy: consider a fraction g of nodes randomly. For each node randomly chosen, immunised their largest degree neighbour.

To compare the strategies, run the epidemics simulations on a Barabasi-Albert network with $N = 1000$ and $m = 3$ with $\beta/\gamma = 0.3$ and $\beta/\gamma = 0.6$, immunised with g ranging $[0, 1]$. Produce a plot where:

- On the x-axis there is g the share of immunised nodes.
- On the y-axis there is $\hat{I}(t = \infty)$, the stationary state of share of infected nodes, as a function of g , for the two immunisation strategies.

Hint: Keep in mind you want to compute the average value of $\hat{I}(t = \infty)$ over multiple run, to avoid small sample bias.

Hint: It may take a while to run the simulations, take this into account.

Look at task_1.4separated.ipynb

task_1.3separated

December 22, 2023

```
[5]: import networkx as nx
import numpy as np
import matplotlib.pyplot as plt
import time
from tqdm import tqdm # Import tqdm
import random
```

```
[6]: def sis_sim(graph, beta, gamma, seed, max_steps=1000):
    random.seed(seed)
    p = random.uniform(0, 1)

    nodes = set(graph.nodes)
    infected_nodes = set(random.sample(nodes, int(p * len(nodes))))
    susceptible_nodes = nodes - infected_nodes

    step = 0
    while step < max_steps:
        infected_nodes_copy = infected_nodes.copy()
        susceptible_nodes_copy = susceptible_nodes.copy()

        for node in susceptible_nodes_copy:
            neighbors = set(graph.neighbors(node))
            infected_neighbors = neighbors & infected_nodes_copy

            num_infected_neighbors = len(infected_neighbors)
            prob_infection = 1 - (1 - beta) ** num_infected_neighbors
            if random.uniform(0, 1) < prob_infection:
                infected_nodes.add(node)
                susceptible_nodes.remove(node)

        for node in infected_nodes_copy:
            if random.uniform(0, 1) < gamma:
                infected_nodes.remove(node)
                susceptible_nodes.add(node)

        step += 1

    return len(infected_nodes) / len(nodes)
```

```
[7]: def average_sis_sim(graph, beta, gamma, sample_size=100, seed=None):
    results = [sis_sim(graph, beta, gamma, seed, max_steps=1000) for _ in
    tqdm(range(sample_size), desc="Average Simulation Progress")]
    return np.mean(results)

def calculate_critical_er_threshold(graph):
    total_degree = sum(dict(graph.degree()).values())
    avg_degree = total_degree / len(graph)
    return 1 / (avg_degree + 1)

def calculate_critical_ba_threshold(graph):
    degrees = dict(graph.degree())
    total_nodes = len(graph)

    avg_degree = sum(degrees.values()) / total_nodes
    avg_square_degree = sum(degree ** 2 for degree in degrees.values()) /
    total_nodes

    return avg_degree / avg_square_degree
```

```
[8]: # Param
beta_gamma_values = np.logspace(-1, 0, 10)
gamma_values = np.logspace(-1, -0.1, 10)
beta_values = beta_gamma_values * gamma_values
sample_size = 100

# Erdos-Renyi
Ner = 500
per = 6 / Ner
er_graph = nx.erdos_renyi_graph(Ner, per)

# Barabasi-Albert
Nba = 500
mba = 3
ba_graph = nx.barabasi_albert_graph(Nba, mba)

# Initialize timer
start_time = time.time()

# Perform sim and calculate averages for ER graph
rho_i_er = []
for beta, gamma in zip(beta_values, gamma_values):
    result_er = average_sis_sim(er_graph, beta, gamma, sample_size)
    rho_i_er.append(result_er)

# Perform sim and calculate averages for BA graph
rho_i_ba = []
```

```

for beta, gamma in zip(beta_values, gamma_values):
    result_ba = average_sis_sim(ba_graph, beta, gamma, sample_size)
    rho_i_ba.append(result_ba)

# Calculate critical thresholds
rc_er = calculate_critical_er_threshold(er_graph)
rc_ba = calculate_critical_ba_threshold(ba_graph)

# Print execution time
end_time = time.time()
execution_time = end_time - start_time
print(f"Execution time: {execution_time:.2f} seconds")

# Plot
plt.figure(figsize=(12, 8))
plt.plot(beta_gamma_values, rho_i_er, label=f' $\rho_I^{ER}(t = \infty)$ ', color='green')
plt.axvline(x=rc_er, color='orange', linestyle='--', label=f'ER Critical  
Threshold RC={rc_er:.2f}')
plt.plot(beta_gamma_values, rho_i_ba, label=f' $\rho_I^{BA}(t = \infty)$ ', color='purple')
plt.axvline(x=rc_ba, color='yellow', linestyle='--', label=f'BA Critical  
Threshold RC={rc_ba:.2f}')

plt.xscale('log')
plt.xlabel(' / ')
plt.ylabel('  $\rho_I$  ')
plt.title('Infected Nodes vs / with the SIS Model')
plt.legend()
plt.show()

```

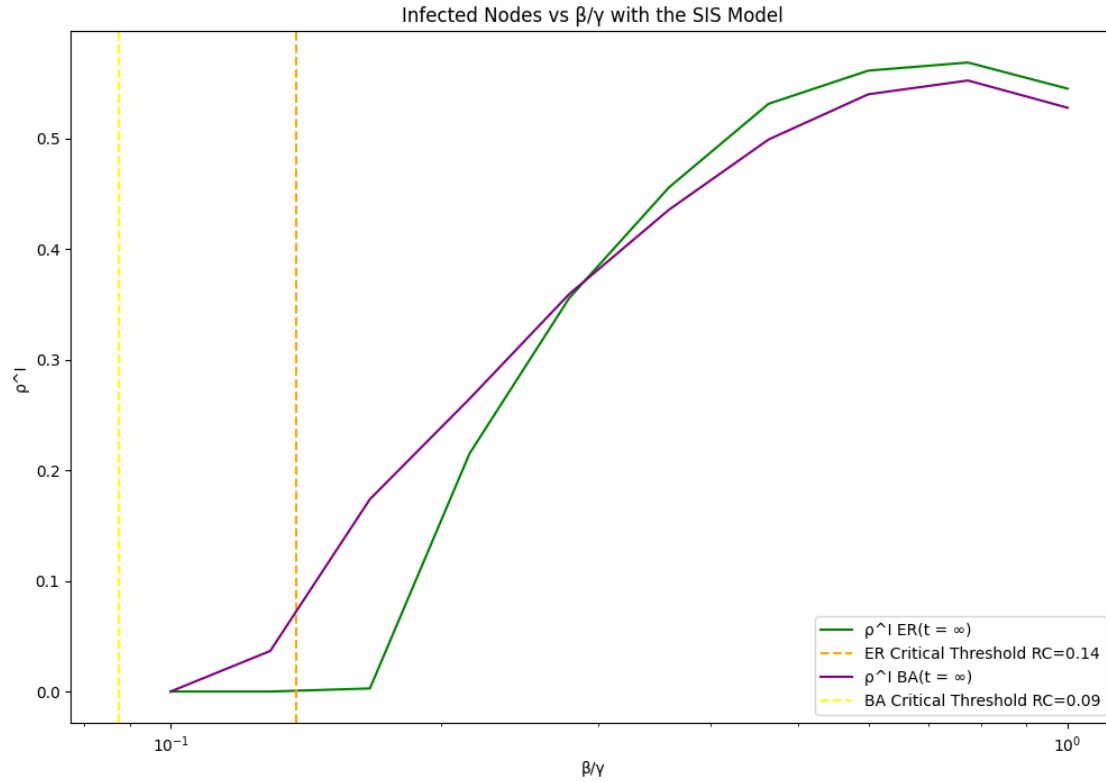
```

Average Simulation Progress: 100%|      | 100/100 [01:05<00:00, 1.52it/s]
Average Simulation Progress: 100%|      | 100/100 [01:06<00:00, 1.51it/s]
Average Simulation Progress: 100%|      | 100/100 [01:10<00:00, 1.41it/s]
Average Simulation Progress: 100%|      | 100/100 [01:05<00:00, 1.52it/s]
Average Simulation Progress: 100%|      | 100/100 [00:58<00:00, 1.71it/s]
Average Simulation Progress: 100%|      | 100/100 [00:54<00:00, 1.84it/s]
Average Simulation Progress: 100%|      | 100/100 [00:48<00:00, 2.06it/s]
Average Simulation Progress: 100%|      | 100/100 [00:48<00:00, 2.07it/s]
Average Simulation Progress: 100%|      | 100/100 [00:50<00:00, 2.00it/s]
Average Simulation Progress: 100%|      | 100/100 [00:53<00:00, 1.86it/s]
Average Simulation Progress: 100%|      | 100/100 [01:06<00:00, 1.49it/s]
Average Simulation Progress: 100%|      | 100/100 [01:09<00:00, 1.45it/s]
Average Simulation Progress: 100%|      | 100/100 [01:04<00:00, 1.56it/s]
Average Simulation Progress: 100%|      | 100/100 [00:58<00:00, 1.70it/s]
Average Simulation Progress: 100%|      | 100/100 [00:54<00:00, 1.85it/s]
Average Simulation Progress: 100%|      | 100/100 [00:52<00:00, 1.89it/s]
Average Simulation Progress: 100%|      | 100/100 [00:48<00:00, 2.07it/s]

```

Average Simulation Progress: 100%| 100/100 [00:49<00:00, 2.02it/s]
Average Simulation Progress: 100%| 100/100 [00:49<00:00, 2.01it/s]
Average Simulation Progress: 100%| 100/100 [00:51<00:00, 1.93it/s]

Execution time: 1147.96 seconds



task_1.4separated

December 22, 2023

```
[11]: import networkx as nx
import numpy as np
import matplotlib.pyplot as plt
from enum import Enum, auto
from tqdm import tqdm
from statistics import mean

[12]: class ImmunisationStrategy(Enum):
    RANDOM = auto()
    TARGETED = auto()

[13]: # Constants
NUM_NODES = 1000
NUM_EDGES = 3
NUM_SIMULATIONS = 100
NUM_TIME_STEPS = 1000
BETA = 0.3
GAMMA = 0.6
G_VALUES = list(np.linspace(0, 1, 10))

[14]: def find_largest_neighbor(graph, node):
    """Finds the largest neighbor of a given node based on degree."""
    neighbors = list(graph.neighbors(node))
    return max(neighbors, key=lambda n: graph.degree(n), default=None) if neighbors else None

[15]: def update_infected_set(network, infected, immune_nodes, infection_rate,
    recovery_rate):
    """Updates the set of infected nodes at each time step."""
    new_infected = infected.copy()
    for infected_node in infected:
        if np.random.rand() < recovery_rate:
            new_infected.remove(infected_node)
        infect_neighbors(network, infected_node, new_infected, immune_nodes,
infection_rate)
    return new_infected
```

```

def simulate_infection_with_immunity(network, infection_rate, recovery_rate,
    ↪initial_infected_node, immune_nodes):
    """Simulates the infection process in the network considering immune nodes.
    ↪"""
    infected = {initial_infected_node}
    probabilities_infected_list = []
    for _ in range(NUM_TIME_STEPS):
        infected = update_infected_set(network, infected, immune_nodes,
    ↪infection_rate, recovery_rate)
        probabilities_infected_list.append(len(infected) / len(network.nodes()))

    return mean(probabilities_infected_list) if probabilities_infected_list
    ↪else 0

def infect_neighbors(network, node, infected, immune_nodes, infection_rate):
    """Attempts to infect the neighbors of a given node."""
    for neighbor in set(network.neighbors(node)).difference(infected,
    ↪immune_nodes):
        if np.random.rand() < infection_rate:
            infected.add(neighbor)

```

```

[16]: def select_immune_nodes(graph, fraction, strategy):
    """Selects a set of immune nodes based on the specified strategy."""
    if not graph.nodes:
        return set()

    graph_nodes = list(graph.nodes())
    np.random.shuffle(graph_nodes)
    num_node_selection = int(fraction * len(graph.nodes()))
    if strategy == ImmunisationStrategy.RANDOM:
        return set(np.random.choice(graph_nodes, num_node_selection,
    ↪replace=False))

    # For TARGETED, avoid calling find_largest_neighbor twice
    return {find_largest_neighbor(graph, node) for node in graph_nodes[:
    ↪num_node_selection] if find_largest_neighbor(graph, node)}

```

```

[17]: def perform_infection_simulations_with_immunity(graph, strategy):
    """Performs multiple infection simulations with a given immunization
    ↪strategy."""
    probabilities_infected = []
    for g in tqdm(G_VALUES, desc=f"Progress {strategy} strategy: "):
        immune_nodes = select_immune_nodes(graph, g, strategy)

```

```

        probabilities_avg = [simulate_infection_with_immunity(graph, BETA,
↪ GAMMA, np.random.choice(list(graph.nodes())), immune_nodes) for _ in
↪ range(NUM_SIMULATIONS)]
        probabilities_infected.append(mean(probabilities_avg))
    return probabilities_infected

```

```

[18]: def generate_barabasi_albert_graph():
        """Generates a Barabási-Albert graph and performs immunization simulations.
        ↪ """
        graph = nx.barabasi_albert_graph(n=NUM_NODES, m=NUM_EDGES)
        immunity_random = perform_infection_simulations_with_immunity(graph,
↪ ImmunisationStrategy.RANDOM)
        immunity_targeted = perform_infection_simulations_with_immunity(graph,
↪ ImmunisationStrategy.TARGETED)
        return immunity_random, immunity_targeted

```

```

[19]: def plot_sis_sim():
        """Plots the results of the SIS model simulations."""
        immunity_random, immunity_targeted = generate_barabasi_albert_graph()
        fig, ax = plt.subplots(figsize=(10, 5))
        ax.set(title="SIS Simulation: Random vs Targeted Immunization", xlabel="g",
↪ ylabel=r"$p^I(t \approx \infty)$", ylim=(0, 1), xlim=(0, 1))
        ax.plot(G_VALUES, immunity_random, label="Random", linestyle='-',
↪ color='blue', marker="o", ms=2)
        ax.plot(G_VALUES, immunity_targeted, label="Targeted", linestyle='--',
↪ color='green', marker="o", ms=2)

```

```

[20]: plot_sis_sim()
plt.show()

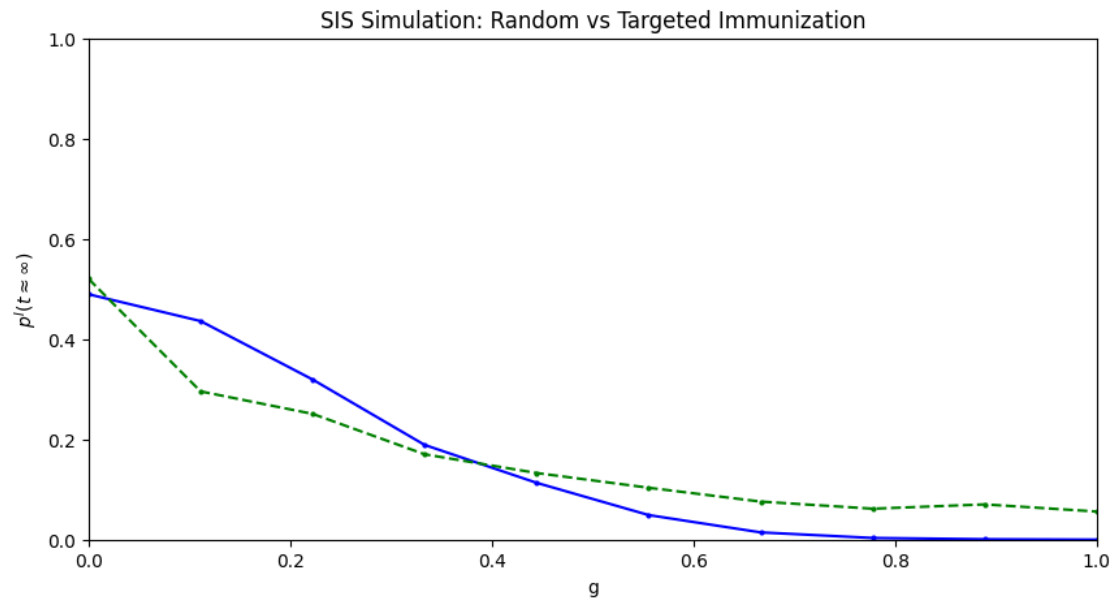
```

```
Progress ImmunisationStrategy.RANDOM strategy: 100%|          | 10/10
```

```
[13:56<00:00, 83.60s/it]
```

```
Progress ImmunisationStrategy.TARGETED strategy: 100%|          | 10/10
```

```
[09:45<00:00, 58.51s/it]
```



Assignment_04

December 22, 2023

1 Authors

- Raphael Wäspi (18-918-938)
- Nicolas Huber (16-936-205)

```
[1093]: import math
# Imports
import numpy as np
import matplotlib.pyplot as plt
import networkx as nx
import os
import glob
import re
import random
import multiprocessing
from statistics import mean

import scipy.special
from tqdm import tqdm
import shutil
from joblib import Parallel, delayed
from scipy.special import factorial
import powerlaw
from NEMtropy import DirectedGraph, UndirectedGraph
from NEMtropy import matrix_generator as mg
from NEMtropy.network_functions import build_adjacency_from_edgelist
from scipy.stats import spearmanr
```

```
[1094]: SEED = 42
```

```
[1095]: # Define colors with reduced opacity from a colormap
COLOR_BLUE = 'dodgerblue'
COLOR_GREEN = 'palegreen'
COLOR_RED = 'lightcoral'
COLOR_YELLOW = 'gold'
COLOR_GREY = 'slategrey'
COLOR_BLACK = 'black'
COLOR_LIGHT_GRAY = 'lightgray'
```

```

COLOR_DARK_GRAY = 'darkgray'

# Define the main data directory
OUTPUT_PATH = "./output"

```

```

[1096]: # Prepare output folder

# Delete the entire 'output' directory and its contents
if os.path.exists(OUTPUT_PATH):
    shutil.rmtree(OUTPUT_PATH)

# Create the 'output' directory
os.makedirs(OUTPUT_PATH)

# Define subdirectories
### OUTPUT_PATH_ERGMS = os.path.join(OUTPUT_PATH, "ERGMS")

# Recreate the subdirectories
### os.makedirs(OUTPUT_PATH_ERGMS)

```

```

[1097]: # Define the directory containing the .gml files
directory = 'datasets'

# Use glob to get a list of all .gml files in the directory
gml_files = glob.glob(os.path.join(directory, '*.gml'))

# Init an empty dictionary for all graphs - the key is the actual name, and the
↳ value is the graph data
graphs = {}

# Use tqdm to show a progress bar in Jupyter Notebook
with tqdm(total=len(gml_files), desc="Loading GML Files") as pbar:
    # Loop through the list of .gml files and load them
    for file_path in gml_files:
        with open(file_path, 'r') as file:
            content = file.read()

            # Use regex to extract the desired part
            match = re.search(r'/(~/)+\.gml$', file_path)
            if match:
                graph_name = match.group(1)

                # Add to the graph dictionary
                graph = nx.read_gml(file_path)
                graphs[graph_name] = graph

```

```

        # Print the graph name when each part is finished
        print(f"Finished loading {graph_name}")

pbar.update(1) # Update the progress bar

```

Loading GML Files: 100%| | 4/4 [00:00<00:00, 27.19it/s]

Finished loading graph_jazz_collab

Finished loading graph1.2

Finished loading graph1.1

Finished loading graph_madrid

2 2. Centralities and influence

2.1 1. Task: Write a function to simulate the SIR dynamic on a generic network and provide visualisation.

The function should take in input: - a network, e.g. if you are using networkx a generic nx.Graph, - the infection rate , - the recovery rate , - an infected seed node.

For the visualisation, you have to plot each of the requested network, using a continous colormap to denote each node Q_i A colorbar for interpretation should be provided.

Graphs to consider: - A Barabasi-Albert network with $N = 100$ and $m = 3$. - A 1D regular lattice with periodic condition, with $N = 100$ nodes and coordination number $k = 6$.

Hint: You have to map the Q_i values to a continous colormap. The colorbar is needed to interpret the plot. Hint: We defined the SIR Q_i in class. In order to have a reliable result, for each node you need to average Q_i over a large sample of simulations.

```

[1100]: def simulate_SIR(network, infection_rate, recovery_rate, infected_seed_node):
        """

        :param network:
        :param infection_rate:
        :param recovery_rate:
        :param infected_seed_node:
        :return:
        """

        # General idea: use makers to denote status of "patients"

        # Initialize node statuses: 0 for susceptible, 1 for infected, 2 for
        ↪ recovered

        # SIR-model: S -> Infection => Infected -> Recovery => Recovered (immune)
        # susceptible state: describes those agents who are in a healthy state, but
        ↪ are prone to become ill

```

```

    # infected state: describes those agents who are under the effects of the
    ↳ disease
    # recovered state: describes those agents who got the disease and cannot
    ↳ transmit it because (e.g.) they recovered, or because they are dead

    # Infection rate Beta: probability to get infected
    # Recovery rate Gamma: probability to recover

    node_statuses = {node: 0 for node in network.nodes()} # Initiate node ->
    ↳ status dict
    node_statuses[infected_seed_node] = 1 # Set the seed node as infected

    susceptible_count = [len(network.nodes()) - 1] # Initial count of
    ↳ susceptible nodes (excluding the seed)
    infected_count = [1] # Initial count of infected nodes
    recovered_count = [0] # Initial count of recovered nodes

    # Simulation loop
    while True:
        # Determine new infections
        new_infections = set()
        for node in network.nodes():
            # For performance reasons
            if node_statuses[node] == 1: # If the node is infected
                neighbors = list(network.neighbors(node))

                # Check all neighbors if an infection happened
                for neighbor in neighbors:
                    if node_statuses[neighbor] == 0 and np.random.random() <
    ↳ infection_rate:
                        new_infections.add(neighbor)

        # Update statuses for new infections
        for node in new_infections:
            node_statuses[node] = 1

        # Determine recoveries
        recoveries = set()
        for node in network.nodes():
            if node_statuses[node] == 1: # If the node is infected
                if np.random.random() < recovery_rate:
                    recoveries.add(node)

        # Update statuses for recoveries
        for node in recoveries:
            node_statuses[node] = 2

```

```

# Count the number of nodes in each status
susceptible_count.append(list(node_statuses.values()).count(0))
infected_count.append(list(node_statuses.values()).count(1))
recovered_count.append(list(node_statuses.values()).count(2))

# Stop the simulation if no new infections or recoveries occur
if len(new_infections) == 0 and len(recoveries) == 0:
    break

# Mapping node statuses to a continuous colormap
node_status_values = [node_statuses[node] for node in network.nodes()]
norm = plt.Normalize(min(node_status_values), max(node_status_values))
cmap = plt.cm.get_cmap('viridis')

# Visualization with continuous colormap
plt.figure(figsize=(10, 6))
ax = plt.subplot(111)
pos = nx.spring_layout(network)
nx.draw(network, pos=pos, node_color=node_status_values,
        cmap=cmap, with_labels=True, node_size=300,
    vmin=min(node_status_values),
        vmax=max(node_status_values), edge_color='gray', width=0.5, ax=ax)

sm = plt.cm.ScalarMappable(cmap=cmap, norm=norm)
sm.set_array([])
plt.colorbar(sm, ax=ax, label='Node Status (0: Susceptible, 1: Infected, 2:
    Recovered)')
plt.title('SIR Simulation on Network with Continuous Colormap')
plt.show()

```

```

[1101]: # Generating Barabasi-Albert network
N_BA = 100 # Number of nodes
m = 3     # Number of edges to attach from a new node to existing nodes
barabasi_albert_graph = nx.barabasi_albert_graph(N_BA, m)

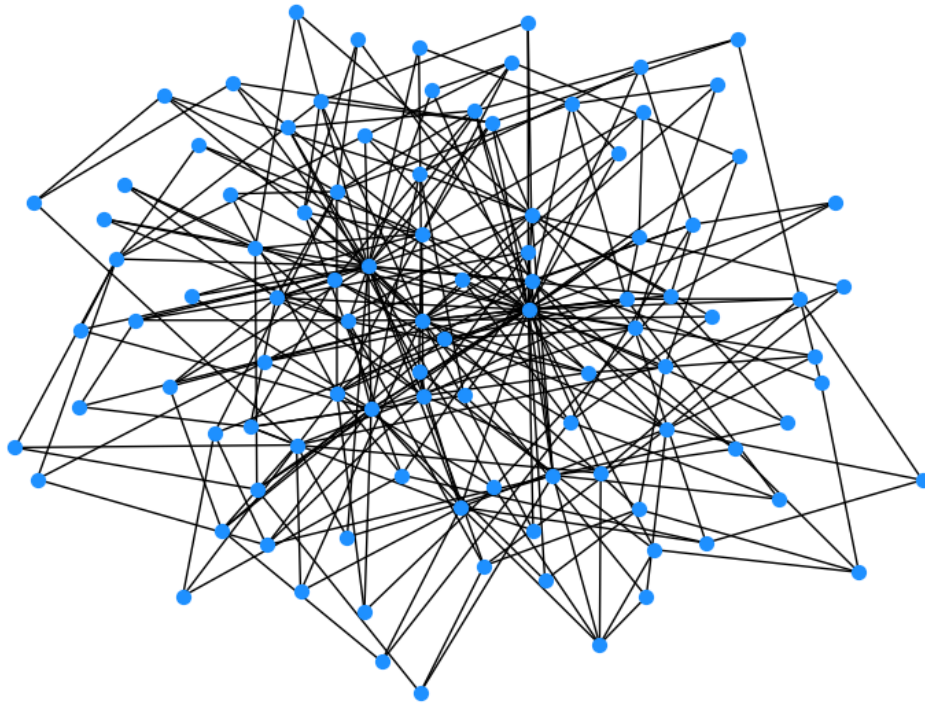
```

```

[1102]: # Visualizing Barabasi-Albert network
plt.figure(figsize=(8, 6))
nx.draw(barabasi_albert_graph, node_size=50, node_color=COLOR_BLUE,
    with_labels=False)
plt.title('Barabasi-Albert Network (N=100, m=3)')
plt.show()

```

Barabasi-Albert Network (N=100, m=3)



```
[1103]: # Generating 1D regular lattice with periodic conditions
N_RL = 100 # Number of nodes
k = 6      # Number of nearest neighbors

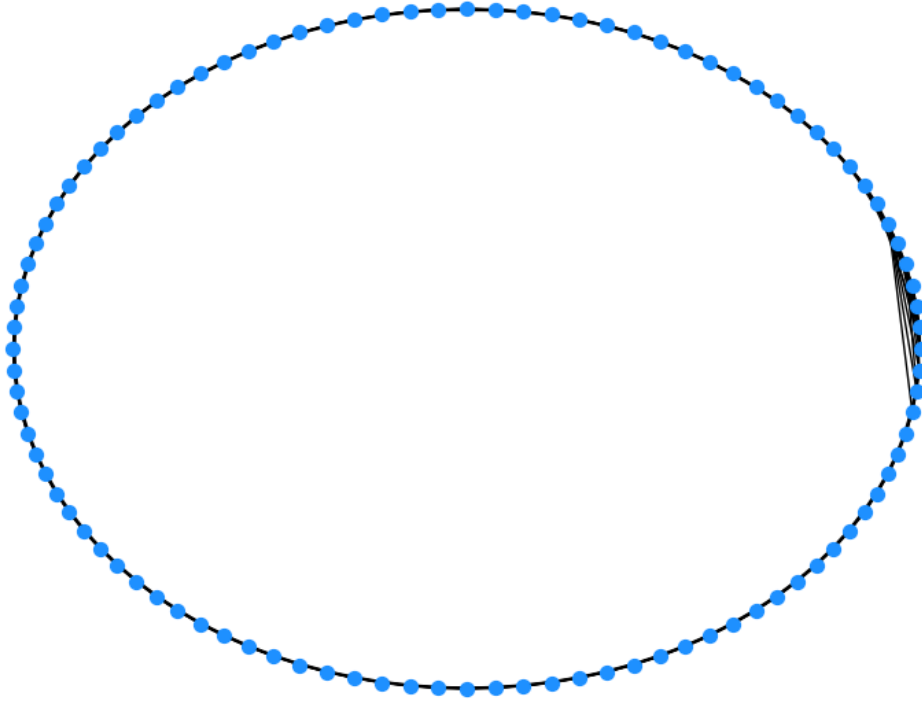
lattice_graph = nx.Graph()

# Create nodes and periodic edges
for i in range(N_RL):
    lattice_graph.add_node(i)
    for j in range(1, k // 2 + 1):
        lattice_graph.add_edge(i, (i + j) % N_RL) # Connect to k/2 nodes on
        ↪ each side
        lattice_graph.add_edge(i, (i - j) % N_RL)

[1104]: # Visualizing 1D regular lattice
plt.figure(figsize=(8, 6))
nx.draw_circular(lattice_graph, node_size=50, node_color=COLOR_BLUE,
        ↪ with_labels=False)
```

```
plt.title('1D Regular Lattice with Periodic Conditions (N=100, k=6)')
plt.show()
```

1D Regular Lattice with Periodic Conditions (N=100, k=6)

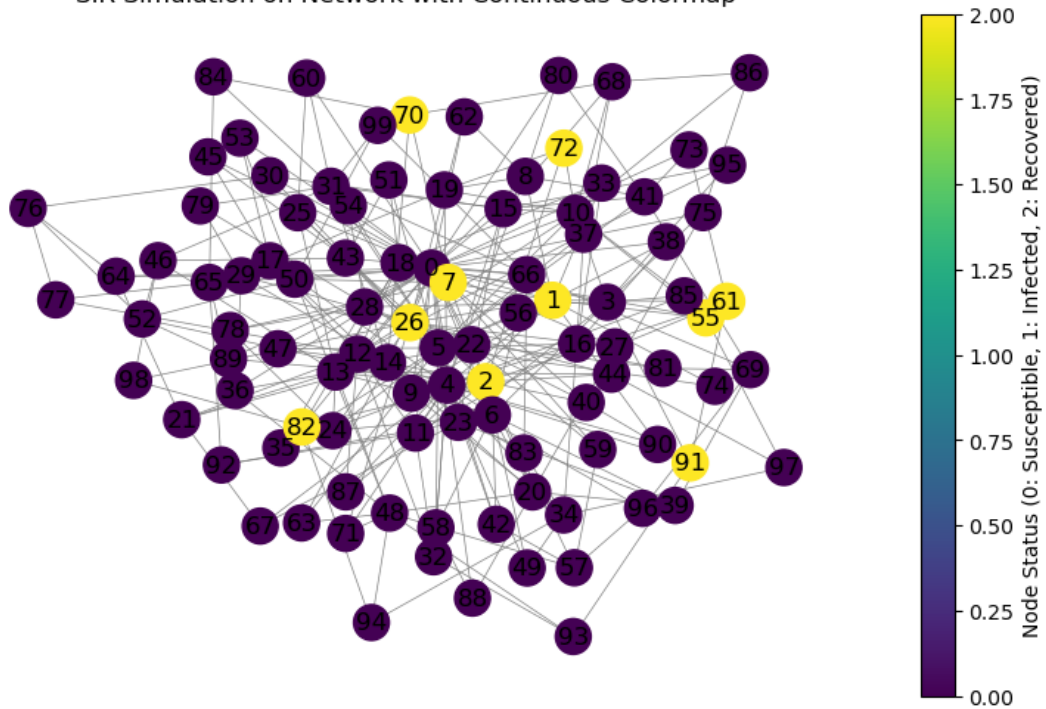


```
[1105]: BETA = 0.2
        GAMMA = 0.6
```

```
[1106]: simulate_SIR(network=barabasi_albert_graph, infection_rate=BETA,
        ↪recovery_rate=GAMMA, infected_seed_node=1)
```

```
/var/folders/ck/vgxn_9f907d122j56f3203gm0000gn/T/ipykernel_22388/607607875.py:73
: MatplotlibDeprecationWarning: The get_cmap function was deprecated in
Matplotlib 3.7 and will be removed two minor releases later. Use
``matplotlib.colormaps[name]`` or ``matplotlib.colormaps.get_cmap(obj)``
instead.
cmap = plt.cm.get_cmap('viridis')
```

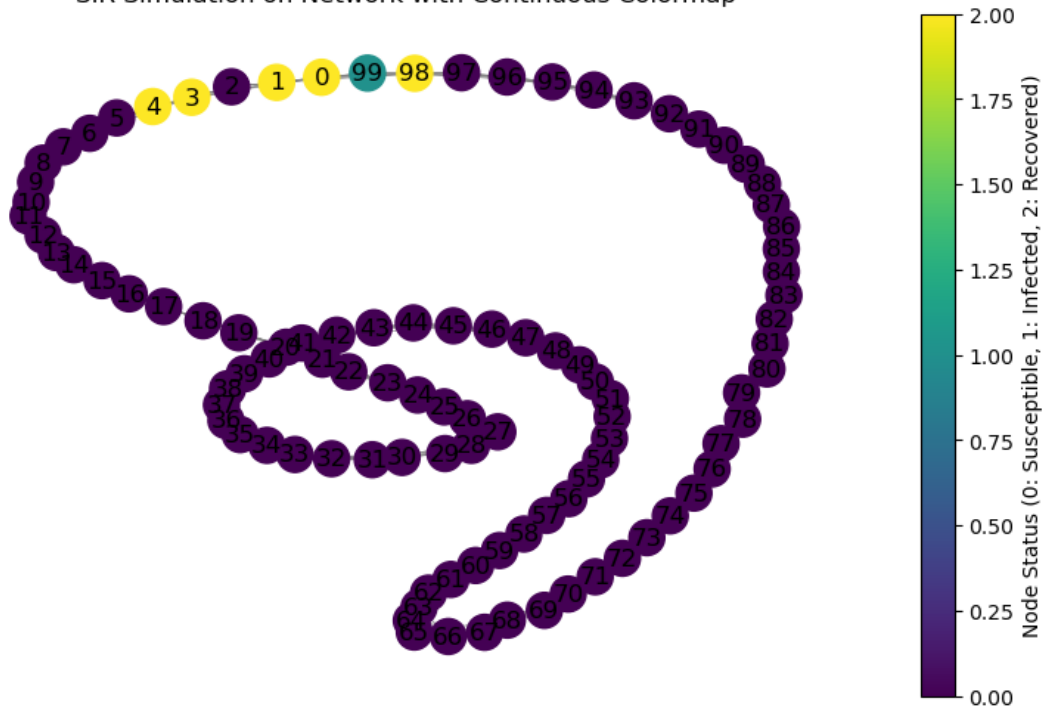
SIR Simulation on Network with Continuous Colormap



```
[1107]: simulate_SIR(network=lattice_graph, infection_rate=BETA, recovery_rate=GAMMA,
↳ infected_seed_node=1)
```

```
/var/folders/ck/vgxn_9f907d122j56f3203gm0000gn/T/ipykernel_22388/607607875.py:73
: MatplotlibDeprecationWarning: The get_cmap function was deprecated in
Matplotlib 3.7 and will be removed two minor releases later. Use
``matplotlib.colormaps[name]`` or ``matplotlib.colormaps.get_cmap(obj)``
instead.
cmap = plt.cm.get_cmap('viridis')
```

SIR Simulation on Network with Continuous Colormap



```
[1108]: def plot_network_state(network, node_statuses, pos, step, step_title):
    # Mapping node statuses to specific colors
    color_map = {0: COLOR_BLUE, 1: COLOR_RED, 2: COLOR_LIGHT_GRAY}

    # Get node colors based on their statuses
    node_colors = [color_map[node_statuses[node]] for node in network.nodes()]

    # Visualization with specific node colors
    plt.figure(figsize=(10, 6))
    ax = plt.subplot(111)
    #pos = nx.spring_layout(network)
    nx.draw(network, pos=pos, node_color=node_colors,
            with_labels=True, node_size=300, edge_color='gray', width=0.5,
    ax=ax)

    # Legend for the colors
    handles = [plt.Line2D([0], [0], marker='o', color='w',
    markerfacecolor=color, markersize=10) for status, color in color_map.items()]
    plt.legend(handles, ['Susceptible', 'Infected', 'Recovered'], loc='best')
    plt.title(f'SIR Simulation on Network with Specific Node Colors -
    {step_title} {step}')
    plt.show()
```

```
[1109]: SAMPLES = 5
        SEED_NODE = 17
        SEED_OFFSET = 42
```

```
[1110]: def simulate_SIR_scenario(network, infection_rate, recovery_rate,
    ↪infected_seed_node, plot=False):
    """

    :param network:
    :param infection_rate:
    :param recovery_rate:
    :param infected_seed_node:
    :return:
    """

    # General idea: use makers to denote status of "patients"

    # Initialize node statuses: 0 for susceptible, 1 for infected, 2 for
    ↪recovered

    # SIR-model: S -> Infection => Infected -> Recovery => Recovered (immune)
    # susceptible state: describes those agents who are in a healthy state, but
    ↪are prone to become ill
    # infected state: describes those agents who are under the effects of the
    ↪disease
    # recovered state: describes those agents who got the disease and cannot
    ↪transmit it because (e.g.) they recovered, or because they are dead

    # Infection rate Beta: probability to get infected
    # Recovery rate Gamma: probability to recover

    node_statuses = {node: 0 for node in network.nodes()} # Initiate node ->
    ↪status dict
    node_statuses[infected_seed_node] = 1 # Set the seed node as infected

    susceptible_count = [len(network.nodes()) - 1] # Initial count of
    ↪susceptible nodes (excluding the seed)
    infected_count = [1] # Initial count of infected nodes
    recovered_count = [0] # Initial count of recovered nodes

    # Generate the initial layout of the network
    pos = nx.spring_layout(network)

    step = 0

    # Initialize simulation
```

```

if plot:
    plot_network_state(network, node_statuses, pos, step, "Initial State")

# Start simulation
# Simulation loop
while True:
    step += 1

    # INFECTIONS
    ###
    # Determine new infections
    new_infections = set()
    for node in network.nodes():
        # For performance reasons
        if node_statuses[node] == 1: # If the node is infected
            neighbors = list(network.neighbors(node))

            # Check all neighbors if an infection happened
            for neighbor in neighbors:
                if node_statuses[neighbor] == 0 and np.random.random() <
↪infection_rate:
                    new_infections.add(neighbor)

    # Update statuses for new infections
    for node in new_infections:
        node_statuses[node] = 1
    ###

    # RECOVERIES
    ###
    # Determine recoveries
    recoveries = set()
    for node in network.nodes():
        if node_statuses[node] == 1: # If the node is infected
            if np.random.random() < recovery_rate:
                recoveries.add(node)

    # Update statuses for recoveries
    for node in recoveries:
        node_statuses[node] = 2
    ###

    # Count the number of nodes in each status
    susceptible_count.append(list(node_statuses.values()).count(0))
    infected_count.append(list(node_statuses.values()).count(1))

```

```

recovered_count.append(list(node_statuses.values()).count(2))

if plot:
    plot_network_state(network, node_statuses, pos, step, f"Step")

# Stop the simulation if no new infections or recoveries occur
if len(new_infections) == 0 and len(recoveries) == 0:
    break

if plot:
    plot_network_state(network, node_statuses, pos, step, f"Final State")

return {
    's_count': susceptible_count,
    'i_count': infected_count,
    'r_count': recovered_count,
    'nodes_statuses': node_statuses
}

```

```

[ ]: simulate_SIR_scenario(network=barabasi_albert_graph, infection_rate=BETA,
    ↪recovery_rate=GAMMA, infected_seed_node=SEED_NODE, plot=False)

```

```

[1112]: def iterate_nodes_with_offset(graph, offset):
    nodes = list(graph.nodes())

    if offset >= 0:
        offset = offset % len(nodes)
        nodes_to_iterate = nodes[offset:] + nodes[:offset]
    else:
        offset = abs(offset) % len(nodes)
        nodes_to_iterate = nodes[-offset:] + nodes[:-offset]

    return nodes_to_iterate

```

```

[1113]: def calculate_average(data):
    # Create a dictionary to store sums for each key
    sum_dict = {}

    # Loop through each dictionary in the data
    for d in data.values():
        for key, value in d.items():
            # Add the value to the sum for the corresponding key
            if key in sum_dict:
                sum_dict[key] += value
            else:

```

```

        sum_dict[key] = value

    # Calculate the average for each key
    num_dicts = len(data)
    avg_dict = {key: value / num_dicts for key, value in sum_dict.items()}

    return avg_dict

```

```

[1114]: def simulate_SIR(network=barabasi_albert_graph, infection_rate=0.1,
    ↪recovery_rate=0.5, plot=False, sample_size=100, seed_start=1):

    rearranged_nodes = iterate_nodes_with_offset(network, seed_start)

    result = {}

    # for all different seeds:
    for node in rearranged_nodes:
        print(node)

        aggregated_values = {}
        results = []

        # simulate for each starting node sample_size times
        for iteration in range(0, sample_size+1, 1):
            pass

            simulation_run =
    ↪simulate_SIR_scenario(network=barabasi_albert_graph,
    ↪infection_rate=infection_rate, recovery_rate=recovery_rate,
    ↪infected_seed_node=node, plot=plot)
            results.append(simulation_run["nodes_statuses"])

        for node_status in results:
            for key, value in node_status.items():
                if key not in aggregated_values:
                    aggregated_values[key] = []
                aggregated_values[key].append(value)

        average_values = {key: sum(values) / len(values) for key, values in
    ↪aggregated_values.items()}
        result[f"starting_node_{node}"]=average_values
        #print(average_values)

    print(result)
    # aggregate and average results

```

```

node_values = calculate_average(result)

return node_values

```

```

[ ]: node_values_ba = simulate_SIR(network=barabasi_albert_graph,
    ↪infection_rate=BETA, recovery_rate=GAMMA, plot=False, sample_size=SAMPLES,
    ↪seed_start=SEED_OFFSET)

```

```

[1116]: # Create a colormap
cmap = plt.cm.get_cmap('viridis')

node_values_ba = list(node_values_ba.values())
plt.figure(figsize=(10, 8))
pos = nx.spring_layout(barabasi_albert_graph)
nodes = nx.draw_networkx_nodes(barabasi_albert_graph, pos=pos,
    ↪node_color=node_values_ba, cmap=cmap, node_size=300, alpha=0.8)
edges = nx.draw_networkx_edges(barabasi_albert_graph, pos=pos,
    ↪edge_color='gray', alpha=0.4)

cbar = plt.colorbar(nodes)
cbar.set_label(fr'$Q_{i}$')
plt.title(fr'SIR simulation of BA-model ({N_BA} nodes): $\beta = {BETA}$,
    ↪$\gamma = {GAMMA}$, samples = {SAMPLES}, offset = {SEED_OFFSET}')
plt.axis('off')
plt.show()

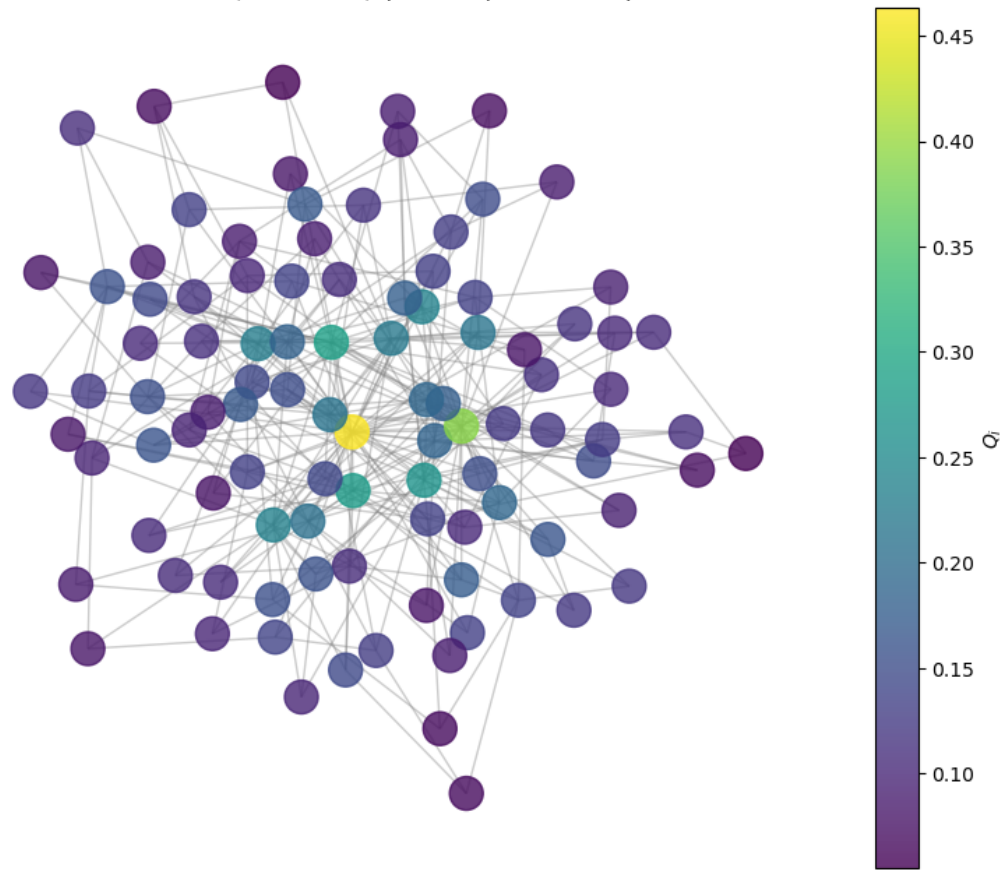
```

```

/var/folders/ck/vgxn_9f907d122j56f3203gm0000gn/T/ipykernel_22388/1748532764.py:2
: MatplotlibDeprecationWarning: The get_cmap function was deprecated in
Matplotlib 3.7 and will be removed two minor releases later. Use
``matplotlib.colormaps[name]`` or ``matplotlib.colormaps.get_cmap(obj)``
instead.
cmap = plt.cm.get_cmap('viridis')

```

SIR simulation of BA-model (100 nodes): $\beta = 0.2$, $\gamma = 0.6$, samples = 5, offset = 42



```
[ ]: node_values_rl = simulate_SIR(network=lattice_graph, infection_rate=BETA,
    ↪recovery_rate=GAMMA, plot=False,
    sample_size=SAMPLES, seed_start=SEED_OFFSET)
```

```
[1118]: # Create a colormap
cmap = plt.cm.get_cmap('viridis')

node_values_rl = list(node_values_rl.values())
plt.figure(figsize=(10, 8))
pos = nx.spring_layout(lattice_graph)
noswa_rl = nx.draw_networkx_nodes(lattice_graph, pos=pos,
    ↪node_color=node_values_rl, cmap=cmap, node_size=300,
    alpha=0.8)
edges = nx.draw_networkx_edges(lattice_graph, pos=pos, edge_color='gray',
    ↪alpha=0.4)

cbar = plt.colorbar(nodes)
cbar.set_label(fr'$Q_{\text{"i"}}$')
```

```
plt.title(
    fr'SIR simulation of BA-model ({N_RL} nodes): $\beta = {BETA}$, $\gamma = {GAMMA}$, samples = {SAMPLES}, offset = {SEED_OFFSET}')
plt.axis('off')
plt.show()
```

```
/var/folders/ck/vgxn_9f907d122j56f3203gm0000gn/T/ipykernel_22388/275618727.py:2:
MatplotlibDeprecationWarning: The get_cmap function was deprecated in Matplotlib
3.7 and will be removed two minor releases later. Use
``matplotlib.colormaps[name]`` or ``matplotlib.colormaps.get_cmap(obj)``
instead.
    cmap = plt.cm.get_cmap('viridis')
```

SIR simulation of BA-model (100 nodes): $\beta = 0.2$, $\gamma = 0.6$, samples = 5, offset = 42



2.2 2. Evaluate the SIR spreading ability of the nodes vs their centrality.

For each of the required network use the SIR function you defined in the previous point to compute the correlation coefficient (Spearman correlation) of the spreading ability Q_i with the following centrality measures: degree, closeness, PageRank and k-core centrality. Plot the correlation coefficient as a function of β / β_C in the range $[0.25, 4]$ β_C , where β_C is the epidemic threshold, with

step size 0.25. Graphs to consider: - Erdos-Renyi with $N=100$ nodes and $p=6/N$. - Barabasi-Albert with $N=100$ nodes and $m=3$ - Watts-Strogatz with $N=100$ nodes, rewiring probability $pr=0.02$ and coordination number $k=6$.

```
[1119]: N = 100
p = 6 / 100
m = 3
pr = 0.02
k = 6
network_er = nx.erdos_renyi_graph(N, p)
network_ba = nx.barabasi_albert_graph(N, m)
network_ws = nx.watts_strogatz_graph(N, k, pr)
```

```
[1120]: def calculate_centralities(graph):
    centralities = {
        "degree": nx.degree_centrality(graph),
        "closeness": nx.closeness_centrality(graph),
        "pagerank": nx.pagerank(graph),
        "kcore": nx.core_number(graph)
    }

    return centralities
```

```
[1121]: # epidemic threshold approx
def epidemic_threshold_approx_sir_model(g):
    #http://networksciencebook.com/chapter/10#network-epidemic

    avg_degree = np.mean([degree for node, degree in g.degree])
    v_degree = np.mean([degree**2 for node, degree in g.degree])

    return avg_degree / v_degree
```

```
[1122]: def x_range(g):
    # Define the range of x values from 0.25 to 4 with 16 points
    x_values = np.linspace(0.25, 4, 16).tolist() # change back second 4 to 16

    # Calculate the scaling factor using epidemic_threshold_approx_sir_model
    ↪function
    scaling = epidemic_threshold_approx_sir_model(g)

    # Scale the x values using the obtained scaling factor
    scaled_x_values = [scaling * x for x in x_values]

    # Define the fixed value for gamma
    gamma_fixed = 0.4 # not sure here
```

```

    # Calculate beta values by multiplying each scaled x value with the fixed
    ↪gamma value
    beta_values = [x * gamma_fixed for x in scaled_x_values]

    return beta_values, [gamma_fixed] * len(scaled_x_values), scaled_x_values

```

```

[1123]: def simulate_SIR(network=barabasi_albert_graph, infection_rate=0.1,
    ↪recovery_rate=0.5, plot=False, sample_size=100, seed_start=1):

    rearranged_nodes = iterate_nodes_with_offset(network, seed_start)

    result = {}

    # for all different seeds:
    for node in rearranged_nodes:
        print(node)

        aggregated_values = {}
        results = []

        # simulate for each starting node sample_size times
        for iteration in range(0, sample_size+1, 1):
            pass

            simulation_run =
    ↪simulate_SIR_scenario(network=barabasi_albert_graph,
    ↪infection_rate=infection_rate, recovery_rate=recovery_rate,
    ↪infected_seed_node=node, plot=plot)
            results.append(simulation_run["nodes_statuses"])

        for node_status in results:
            for key, value in node_status.items():
                if key not in aggregated_values:
                    aggregated_values[key] = []
                    aggregated_values[key].append(value)

        average_values = {key: sum(values) / len(values) for key, values in
    ↪aggregated_values.items()}
        result[f"starting_node_{node}"]=average_values
        #print(average_values)

    #print(result)
    # aggregate and average results
    node_values = calculate_average(result)

```

```
return node_values
```

```
[1124]: nets = {
    'er': {
        'name': 'Erdos Reigny',
        'graph': network_er,
        'centralities': calculate_centralities(network_er),
        'betas': x_range(network_er)[0],
        'gammas': x_range(network_er)[1],
        'x_values': x_range(network_er)[2],
        'results': {
            'spreading': { },
            'correlations': {
                "degree": [],
                "closeness": [],
                "pagerank": [],
                "kcore": [],
            }
        },
    },
    'ba': {
        'name': 'Barabasi-Albert',
        'graph': network_ba,
        'centralities': calculate_centralities(network_ba),
        'betas': x_range(network_ba)[0],
        'gammas': x_range(network_ba)[1],
        'x_values': x_range(network_ba)[2],
        'results': {
            'spreading': { },
            'correlations': {
                "degree": [],
                "closeness": [],
                "pagerank": [],
                "kcore": [],
            }
        },
    },
    'ws': {
        'name': 'Watts-Strogatz',
        'graph': network_ws,
        'centralities': calculate_centralities(network_ws),
        'betas': x_range(network_ws)[0],
        'gammas': x_range(network_ws)[1],
        'x_values': x_range(network_ws)[2],
        'results': {
            'spreading': { },
            'correlations': {
```

```

        "degree": [],
        "closeness": [],
        "pagerank": [],
        "kcore": [],
    }
},
},
}

```

```

[ ]: ba_net = nets['ba'] # Accessing the 'ba' network dictionary

ba_results = {}
for beta, gamma, x in zip(ba_net['betas'], ba_net['gammas'],
    ↪ba_net['x_values']):
    node_values_ba = simulate_SIR(network=ba_net['graph'], infection_rate=beta,
    ↪recovery_rate=gamma, plot=False, sample_size=5, seed_start=0)
    ba_results[x] = node_values_ba

nets['ba']['results']['spreading'] = ba_results

```

```

[ ]: er_net = nets['er']

er_results = {}
for beta, gamma, x in zip(er_net['betas'], er_net['gammas'],
    ↪er_net['x_values']):
    node_values_er = simulate_SIR(network=er_net['graph'], infection_rate=beta,
    ↪recovery_rate=gamma, plot=False, sample_size=5, seed_start=0)
    er_results[x] = node_values_er

nets['er']['results']['spreading'] = er_results

```

```

[ ]: ws_net = nets['ws']

ws_results = {}
for beta, gamma, x in zip(ws_net['betas'], ws_net['gammas'],
    ↪ws_net['x_values']):
    node_values_ws = simulate_SIR(network=ws_net['graph'], infection_rate=beta,
    ↪recovery_rate=gamma, plot=False, sample_size=5, seed_start=0)
    ws_results[x] = node_values_ws

nets['ws']['results']['spreading'] = ws_results

```

```

[ ]: nets['ba']

```

```

[ ]: for graph, graph_data in nets.items():
    print(graph)

```

```

spreading_values = graph_data['results']['spreading']

for x_value, spreading_nodes in spreading_values.items():
    #corrs = []
    c_key = ''
    #print(graph_data['centralities'])
    for centrality, centrality_values in graph_data['centralities'].items():
        c_key = centrality
        spearman_corr, _ = spearmanr(list(centrality_values.values()),
↪list(spreading_nodes.values()))
        print(centrality, x_value, spearman_corr)
        graph_data['results']['correlations'][c_key].append(spearman_corr)

    #print(x_value, len(spreading_nodes))

```

```
[ ]: nets
```

```

[1132]: # Calculate degree centrality of the graph
degree_centrality = nx.degree_centrality(network_ba)
closeness_centrality = nx.closeness_centrality(network_ba)
k_core_centrality = nx.core_number(network_ba)
page_rank_centrality = nx.pagerank(network_ba)

centrality_measures_ba = {
    'Degree': degree_centrality,
    'Closeness': closeness_centrality,
    'K-Core': k_core_centrality,
    'PageRank': page_rank_centrality
}

```

```

[1133]: x_values = list(ba_results.keys())
correlation_results = {measure_name: [] for measure_name in
↪centrality_measures_ba}

```

```

[1134]: for measure_name, measure_values in centrality_measures_ba.items():
        print(f"Calculating Spearman's rank correlation for {measure_name}
↪centrality...")
        for x_values, nodes_spreading in ba_results.items():
            dict_values = [nodes_spreading[node] for node in network_ba.nodes() if
↪node in measure_values]
            centrality_values = [measure_values[node] for node in network_ba.
↪nodes() if node in measure_values]

```

```

spearman_corr, _ = spearmanr(dict_values, centrality_values)
#print(f"Spearman's rank correlation coefficient for {measure_name}:␣
↪{spearman_corr}")
correlation_results[measure_name].append(spearman_corr)

```

Calculating Spearman's rank correlation for Degree centrality...

Calculating Spearman's rank correlation for Closeness centrality...

Calculating Spearman's rank correlation for K-Core centrality...

Calculating Spearman's rank correlation for PageRank centrality...

```
[1135]: print(correlation_results)
```

```

{'Degree': [0.14198654649882947, 0.28307444396439113, 0.3741494518072024,
0.43577097052157887, 0.42695528144850076, 0.4991376884169361, 0.521574287207661,
0.5301665012894126, 0.55735710205435, 0.524361790429919, 0.5011392320770367,
0.5041617622706598, 0.6240649836694157, 0.5442053486561761, 0.5380313658128348,
0.608772200289035], 'Closeness': [0.18399751593198874, 0.3725930797758298,
0.36532539821464155, 0.5181069890153414, 0.5346638729153492, 0.5824890716420497,
0.5287651660328336, 0.6546020472063725, 0.6399152491380712, 0.681516545108969,
0.5947436084938673, 0.6340621048601456, 0.7095614481889252, 0.6796853793491301,
0.6631354484838264, 0.6666676825693869], 'K-Core': [nan, nan, nan, nan, nan,
nan, nan, nan, nan, nan, nan, nan, nan, nan, nan], 'PageRank':
[0.11822900179675164, 0.24843728816328886, 0.3459603287270973,
0.37265089266188456, 0.37181061204966426, 0.427680291547995,
0.48582627928267647, 0.466731086443845, 0.4964247909666162, 0.4417278100615553,
0.4506856320957219, 0.43890774144316547, 0.5644460058192742,
0.47901370908416585, 0.47574356507628834, 0.548485430407701]}

```

```

[1136]: def plot_correlations(nets):
    fig, axs = plt.subplots(4, 1, figsize=(10, 12))
    correlations = ["degree", "closeness", "pagerank", "kcore"]

    for net_type, data in nets.items():
        x_values = data['x_values']
        results = data['results']['correlations']

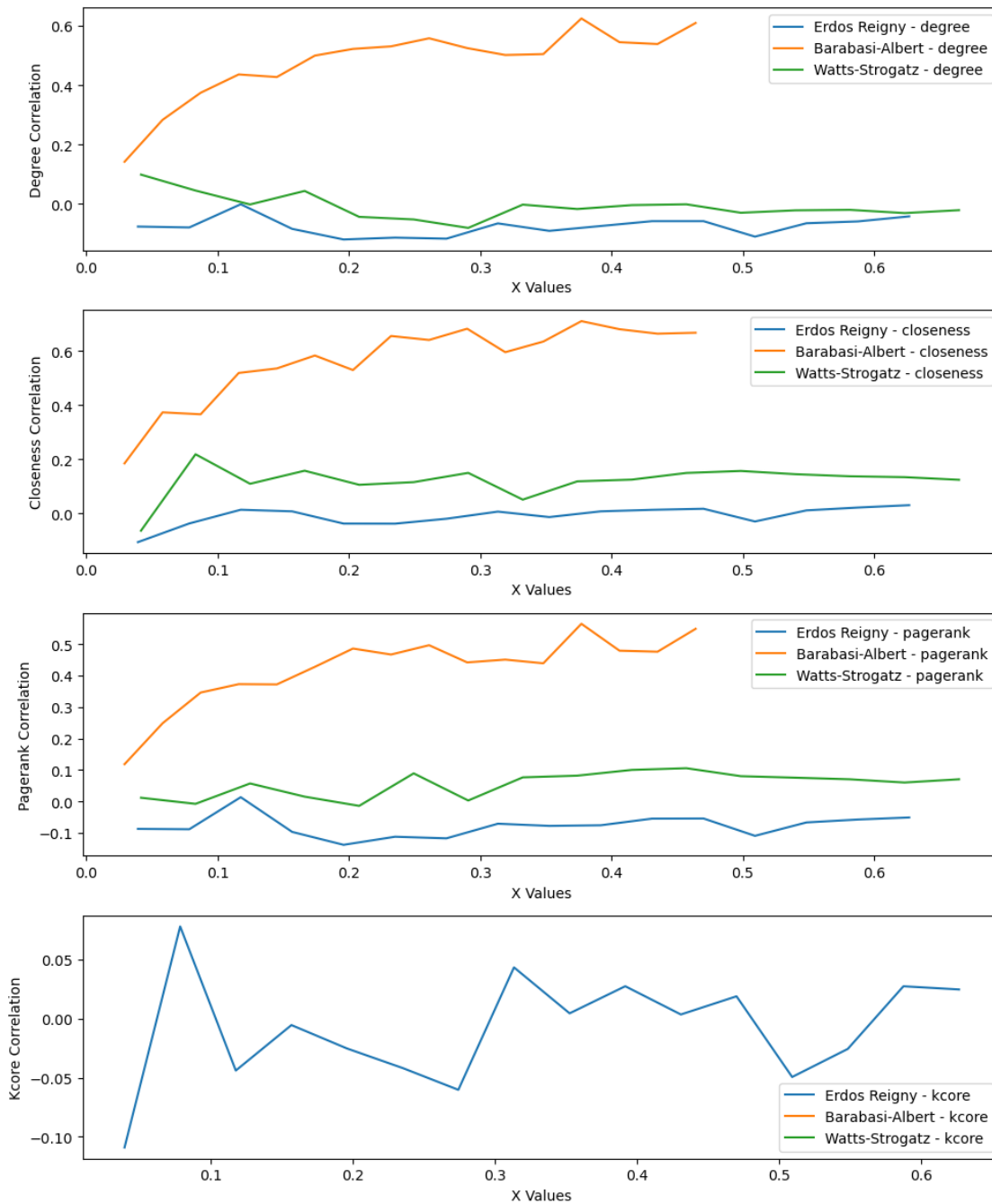
        for i, correlation in enumerate(correlations):
            axs[i].plot(x_values, results[correlation], label=f"{data['name']}␣
↪- {correlation}")

        for i, correlation in enumerate(correlations):
            axs[i].set_xlabel('X Values')
            axs[i].set_ylabel(f'{correlation.capitalize()} Correlation')
            axs[i].legend()

    plt.tight_layout()
    plt.show()

```

```
[1137]: plot_correlations(nets)
```



2.3 T3 Produce the scatter plot of centrality measure vs spreading ability.

Do a scatter plot of Q_{ivs} each centrality measure x_i from the previous task, at the critical point $= C_{by}$ by normalizing the maximum value to one as $Q/\max(Q)$ and $x/\max(x)$, respectively. Graphs to consider: • Erdos-Renyi with $N=100$ nodes and $p=6/N$. • Barabasi-Albert with $N=100$ nodes and $m=$

3. • Watts-Strogatz with $N=100$ nodes, rewiring probability $p_r=0.02$ and coordination number $k=6$.

```
[1138]: def adapted_simulate_SIR(G, beta, gamma, time_steps, sample_size):
    nodes = list(G.nodes())
    results = {}

    for node in nodes:
        aggregated_values = []

        for _ in range(sample_size):
            set_infected = {node}
            set_susceptible = set(G) - set_infected
            set_recovered = set()

            node_neighbors = {n: set(G.neighbors(n)) for n in G}

            for _ in range(time_steps):
                newly_infected = set()
                newly_recovered = set()

                for infected_node in set_infected:
                    neighbors = node_neighbors[infected_node]
                    for neighbor in neighbors:
                        if neighbor in set_infected or neighbor in newly_infected:
                            continue
                        if np.random.rand() < beta:
                            newly_infected.add(neighbor)
                if np.random.rand() < gamma:
                    newly_recovered.add(infected_node)

                set_susceptible -= newly_infected
                set_infected |= newly_infected
                set_infected -= newly_recovered
                set_recovered |= newly_recovered

            aggregated_values.append(len(set_recovered) / G.number_of_nodes())

        results[f"starting_node_{node}"] = np.mean(aggregated_values)

    return results
```

```
[1139]: def calculate_network_centrality_measures(graph):
    degree_centrality = nx.degree_centrality(graph)
    closeness_centrality = nx.closeness_centrality(graph)
    pagerank_scores = nx.pagerank(graph)
    k_core_numbers = nx.core_number(graph)
```

```

    return degree_centrality, closeness_centrality, pagerank_scores,
    ↪k_core_numbers

```

```

[1140]: def normalize_centrality_values(centrality_dict):
        values_array = np.array(list(centrality_dict.values()))
        return values_array / values_array.max()

```

```

[1141]: def plot_normalized_centrality_vs_spreading_ability(spreading_ability, degree,
    ↪closeness, pagerank, k_core, graph_title):
        plt.scatter(x=spreading_ability, y=degree, label='Degree Centrality',
    ↪alpha=0.2)
        plt.scatter(x=spreading_ability, y=closeness, label='Closeness Centrality',
    ↪alpha=0.2)
        plt.scatter(x=spreading_ability, y=pagerank, label='PageRank Centrality',
    ↪alpha=0.2)
        plt.scatter(x=spreading_ability, y=k_core, label='K-Core Centrality',
    ↪alpha=0.2)
        plt.xlabel('$Q_{i}$ (Normalized Spreading Ability)', fontsize=15)
        plt.ylabel('Normalized Centrality Measure', fontsize=13)
        plt.title(graph_title)
        plt.legend()
        plt.show()

```

```

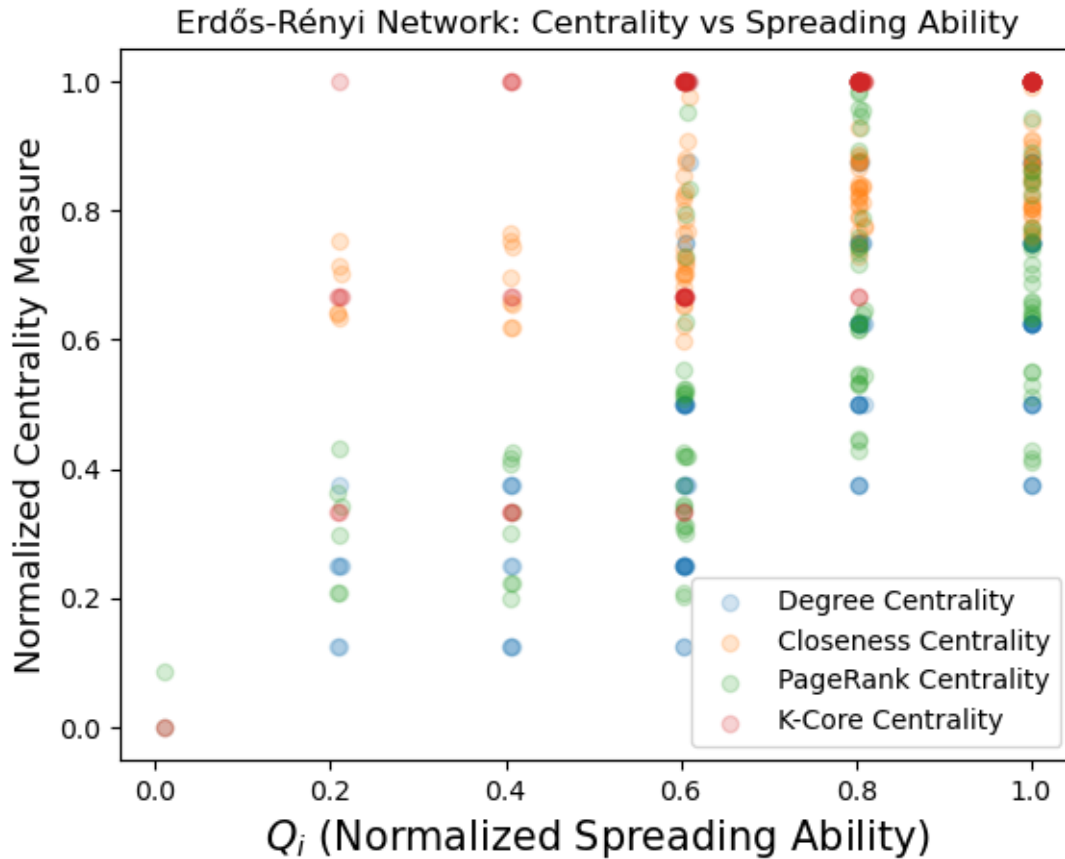
[1142]: erdos_renyi_graph = nx.erdos_renyi_graph(n=100, p=0.05)
        degree_cent, closeness_cent, pagerank_cent, k_core_cent =
    ↪calculate_network_centrality_measures(erdos_renyi_graph)

        # Computing and normalizing spreading ability
        spreading_ability_results = adapted_simulate_SIR(erdos_renyi_graph, beta=0.1,
    ↪gamma=0.1, time_steps=500, sample_size=5)
        normalized_spreading_ability =
    ↪normalize_centrality_values(spreading_ability_results)

        # Normalizing centrality measures
        normalized_degree_cent = normalize_centrality_values(degree_cent)
        normalized_closeness_cent = normalize_centrality_values(closeness_cent)
        normalized_pagerank_cent = normalize_centrality_values(pagerank_cent)
        normalized_k_core_cent = normalize_centrality_values(k_core_cent)

        # Plotting
        plot_normalized_centrality_vs_spreading_ability(normalized_spreading_ability,
    ↪normalized_degree_cent, normalized_closeness_cent, normalized_pagerank_cent,
    ↪normalized_k_core_cent, 'Erdős-Rényi Network: Centrality vs Spreading
    ↪Ability')

```



```
[1143]: barabasi_albert_graph = nx.barabasi_albert_graph(n=100, m=3)
degree_cent, closeness_cent, pagerank_cent, k_core_cent = □
↳ calculate_network_centralities_measures(barabasi_albert_graph)

# Computing and normalizing spreading ability
spreading_ability_results = adapted_simulate_SIR(barabasi_albert_graph, beta=0.
↳ 1, gamma=0.1, time_steps=500, sample_size=5)
normalized_spreading_ability = □
↳ normalize_centralities_values(spreading_ability_results)

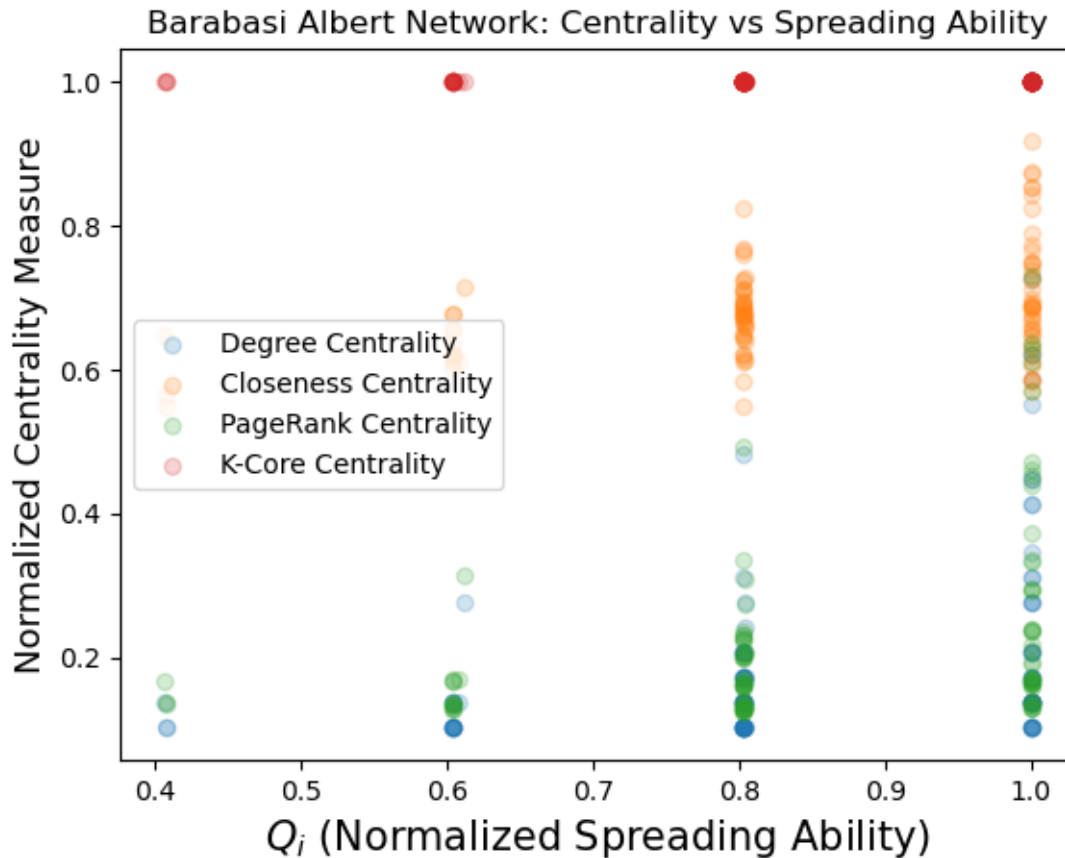
# Normalizing centrality measures
normalized_degree_cent = normalize_centralities_values(degree_cent)
normalized_closeness_cent = normalize_centralities_values(closeness_cent)
normalized_pagerank_cent = normalize_centralities_values(pagerank_cent)
normalized_k_core_cent = normalize_centralities_values(k_core_cent)

# Plotting
```

```

plot_normalized_centrality_vs_spreading_ability(normalized_spreading_ability,
↳normalized_degree_cent, normalized_closeness_cent, normalized_pagerank_cent,
↳normalized_k_core_cent, 'Barabasi Albert Network: Centrality vs Spreading
↳Ability')

```



```

[1144]: watts_strogatz_graph = nx.watts_strogatz_graph(n=100, k=6, p=0.02)
degree_cent, closeness_cent, pagerank_cent, k_core_cent =
↳calculate_network_centrality_measures(watts_strogatz_graph)

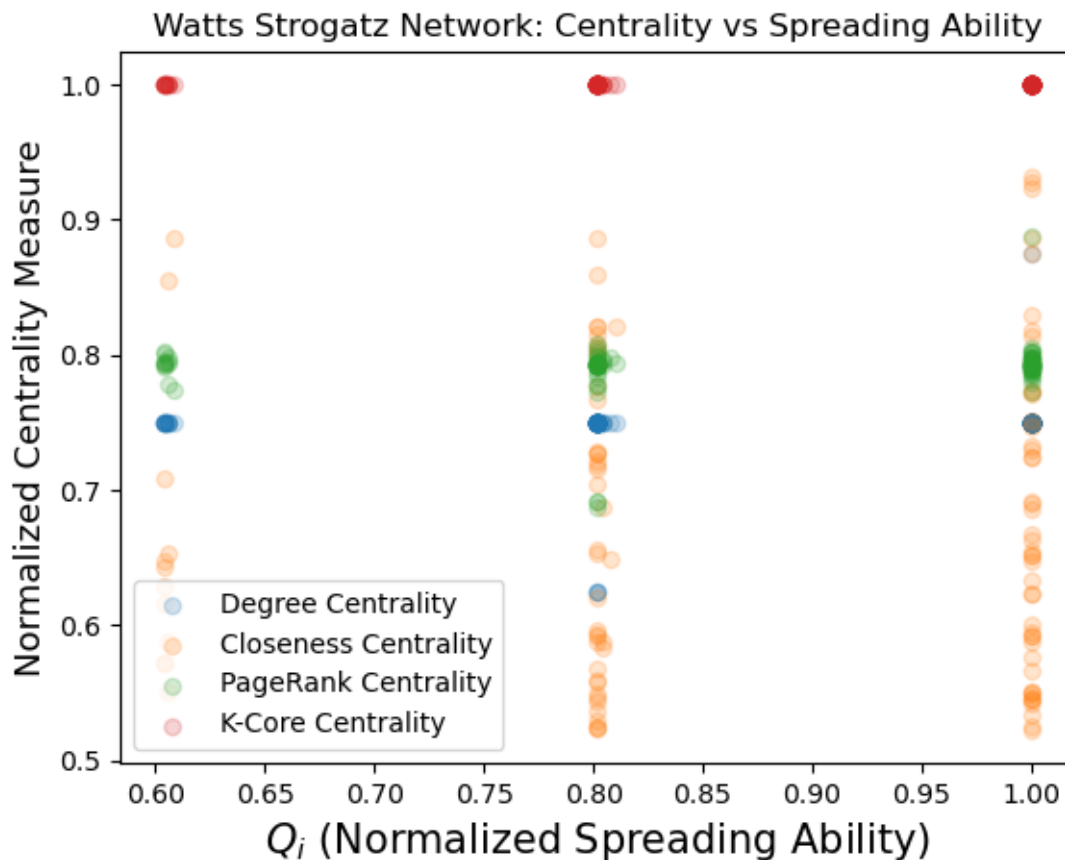
# Computing and normalizing spreading ability
spreading_ability_results = adapted_simulate_SIR(watts_strogatz_graph, beta=0.
↳1, gamma=0.1, time_steps=500, sample_size=5)
normalized_spreading_ability =
↳normalize_centrality_values(spreading_ability_results)

# Normalizing centrality measures
normalized_degree_cent = normalize_centrality_values(degree_cent)
normalized_closeness_cent = normalize_centrality_values(closeness_cent)
normalized_pagerank_cent = normalize_centrality_values(pagerank_cent)

```

```
normalized_k_core_cent = normalize_centrality_values(k_core_cent)

# Plotting
plot_normalized_centrality_vs_spreading_ability(normalized_spreading_ability,
    ↪normalized_degree_cent, normalized_closeness_cent, normalized_pagerank_cent,
    ↪normalized_k_core_cent, 'Watts Strogatz Network: Centrality vs Spreading Ability',
    ↪Ability')
```



2.4 T4

```
[1145]: graphs
```

```
[1145]: {'graph_jazz_collab': <networkx.classes.graph.Graph at 0x1339dfb90>,
        'graph1.2': <networkx.classes.graph.Graph at 0x12da74990>,
        'graph1.1': <networkx.classes.graph.Graph at 0x12db0fcd0>,
        'graph_madrid': <networkx.classes.graph.Graph at 0x1339f2410>}
```

```
[1146]: madrid_graph = nx.read_gml('./datasets/graph_madrid.gml')
        jazz_collab_graph = nx.read_gml('./datasets/graph_jazz_collab.gml')
```

```

print("Madrid graph is a", type(madrid_graph))
print("Node labels in Madrid graph:", list(madrid_graph.nodes(data=True))[0:5])

print("Jazz Collaboration graph is a", type(jazz_collab_graph))
print("Node labels in Jazz Collaboration graph:", list(jazz_collab_graph.
    ↪nodes(data=True))[0:5])
# Conversion
madrid_graph = madrid_graph.to_undirected() if not isinstance(madrid_graph, nx.
    ↪Graph) else madrid_graph
jazz_collab_graph = jazz_collab_graph.to_undirected() if not
    ↪isinstance(jazz_collab_graph, nx.Graph) else jazz_collab_graph

```

Madrid graph is a <class 'networkx.classes.graph.Graph'>

Node labels in Madrid graph: [('1', {}), ('2', {}), ('3', {}), ('4', {}), ('5', {})]

Jazz Collaboration graph is a <class 'networkx.classes.graph.Graph'>

Node labels in Jazz Collaboration graph: [('0', {}), ('1', {}), ('2', {}), ('3', {}), ('4', {})]

```

[1147]: # Ensure the graphs are undirected, as SIR simulation typically runs on
    ↪undirected graphs

```

```

madrid_graph = graphs['graph_madrid'].to_undirected()
jazz_collab_graph = graphs['graph_jazz_collab'].to_undirected()

```

```

[1148]: # Perform the analysis on the Madrid graph

```

```

madrid_spreading_ability = adapted_simulate_SIR(madrid_graph, beta=0.1, gamma=0.
    ↪1, time_steps=500, sample_size=5)

```

```

madrid_degree, madrid_closeness, madrid_pagerank, madrid_kcore =
    ↪calculate_network_centrality_measures(madrid_graph)

```

```

# Normalize the centrality values

```

```

madrid_degree_norm = normalize_centrality_values(madrid_degree)
madrid_closeness_norm = normalize_centrality_values(madrid_closeness)
madrid_pagerank_norm = normalize_centrality_values(madrid_pagerank)
madrid_kcore_norm = normalize_centrality_values(madrid_kcore)

```

```

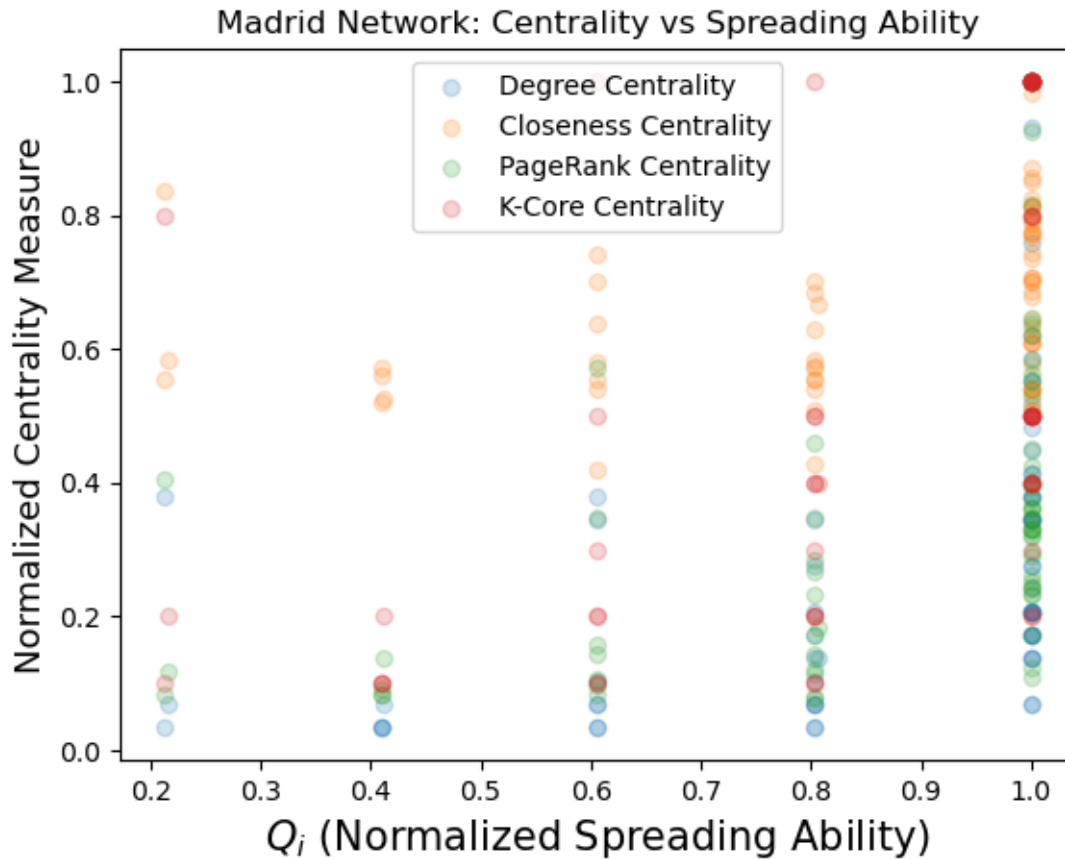
# Plotting the results for the Madrid graph

```

```

plot_normalized_centrality_vs_spreading_ability(
    normalize_centrality_values(madrid_spreading_ability),
    madrid_degree_norm,
    madrid_closeness_norm,
    madrid_pagerank_norm,
    madrid_kcore_norm,
    'Madrid Network: Centrality vs Spreading Ability'
)

```

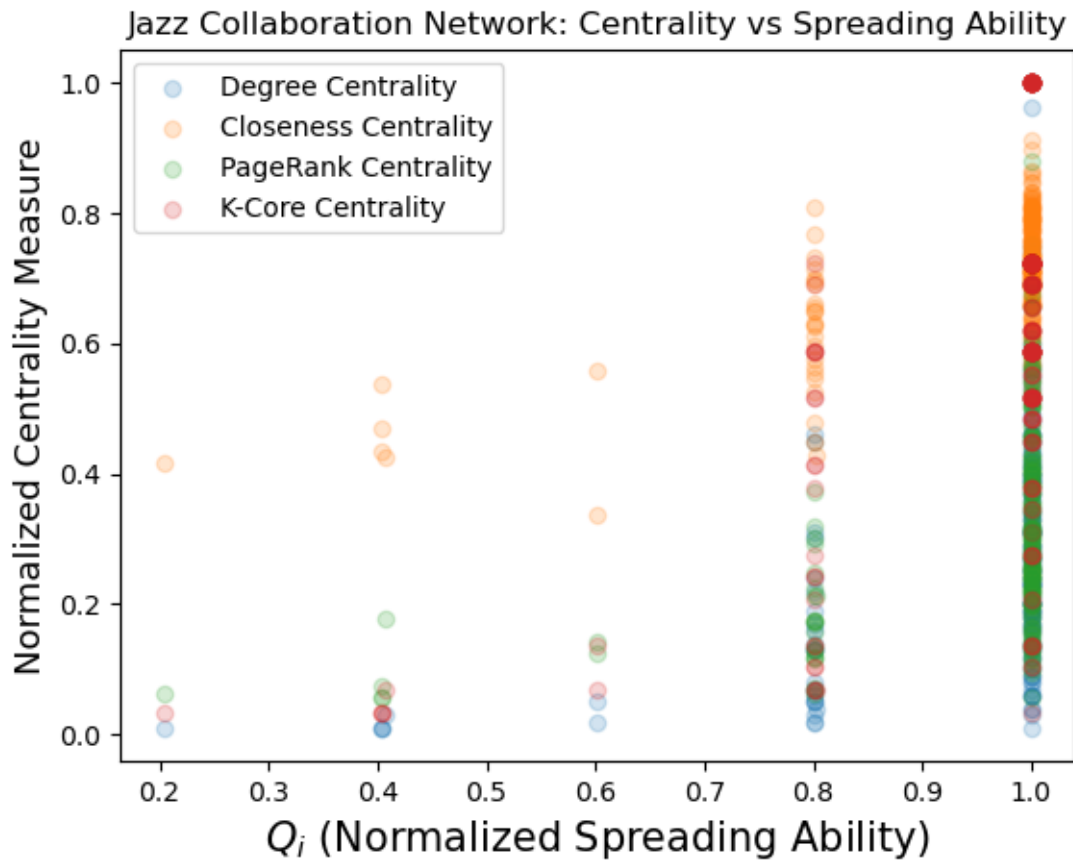


```
[1149]: # Perform the analysis on the Jazz Collaboration graph
jazz_spreading_ability = adapted_simulate_SIR(jazz_collab_graph, beta=0.1,
    ↪gamma=0.1, time_steps=500, sample_size=5)
jazz_degree, jazz_closeness, jazz_pagerank, jazz_kcore =
    ↪calculate_network Centrality Measures(jazz_collab_graph)

# Normalize the centrality values
jazz_degree_norm = normalize Centrality Values(jazz_degree)
jazz_closeness_norm = normalize Centrality Values(jazz_closeness)
jazz_pagerank_norm = normalize Centrality Values(jazz_pagerank)
jazz_kcore_norm = normalize Centrality Values(jazz_kcore)

# Plotting the results for the Jazz Collaboration graph
plot_normalized_Centrality_vs_Spreading_Ability(
    normalize Centrality Values(jazz_spreading_ability),
    jazz_degree_norm,
    jazz_closeness_norm,
    jazz_pagerank_norm,
    jazz_kcore_norm,
```

'Jazz Collaboration Network: Centrality vs Spreading Ability'



Overall: - Examine if centrality correlates with spreading ability -> more central individuals might also be those who can spread influence more effectively. - Consider the structural differences between the networks that may affect the patterns observed. Madrid Network: - Centrality and Influenced: High centrality can probably indicate key figures in the network (hubs), which potentially involved in operational roles or information dissemination within the group. - Spreading Ability: Important for understanding the network's resilience and identifying critical nodes. Jazz Collaboration Network - Centrality and Influence: Centrality could reflect musicians' prominence or their role as connectors in the jazz community. - Spreading Ability: May represent a musician's capacity to spread and influence new musical styles or songs through collaborations.